



รายงานปฏิบัติการ PS07: Concurrency and Thread

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ภาพรวมของโปรแกรม	1
1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน	1
1.2 โครงสร้างของโปรแกรม	1
1.3 การคอมไพล์	3
2 Task 1: รันด้วย thread เดียว	3
2.1 การคอมไพล์และสภาพแวดล้อม	3
2.2 ผลการรันด้วย thread เดียว	4
3 Task 2: หลาย thread โดยไม่บังคับ context switch	4
3.1 รันบน CPU 22 ตัว	4
3.2 รันบน CPU ตัวเดียวด้วย taskset	5
3.3 ทำไม $balance = balance + AMOUNT$ จึงไม่ปลอดภัย	6
3.4 เปรียบเทียบผลบน CPU 22 ตัวกับ CPU ตัวเดียว	7

สภาพแวดล้อมที่ใช้ในการทดลอง

การทดลองทั้งหมดทำบน Linux ที่ทำงานอยู่บน WSL2 ของเครื่องคอมพิวเตอร์เครื่องเดียว จุดที่มีผลต่อการทดลองนี้โดยตรงคือจำนวน CPU เพราะ thread ที่รันบน CPU คนละแกนทำงานพร้อมกันได้จริง ไม่ต้องรอให้ระบบปฏิบัติการสลับให้ รายละเอียดของสภาพแวดล้อมเป็นดังนี้

Table 1: ข้อมูลสภาพแวดล้อมของระบบที่ใช้ในการทดลอง

รายการ	รายละเอียด
ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS ทำงานบน WSL2 ของ Windows 11 Home Single Language (build 26200)
Kernel	6.6.87.2-microsoft-standard-WSL2 (x86-64, SMP PREEMPT_DYNAMIC)
CPU	Intel Core Ultra 9 185H ที่ Linux มองเห็นเป็น 22 logical CPU (11 core แกนละ 2 thread)
คอมไพเลอร์	gcc 13.3.0 ไม่ระบุระดับ optimization จึงใช้ค่าเริ่มต้นคือ -O0
Thread library	POSIX threads ของ glibc 2.39 (ลิงก์ด้วย -lpthread)
เครื่องมือเสริม	taskset (util-linux) สำหรับบังคับให้ process ทำงานบน CPU ที่กำหนด

หมายเหตุเรื่องหลักฐาน: ผลการรันทุกครั้งที่แสดงในรายงานเป็นภาพหน้าจอจริงจาก terminal ตัวเลขทุกตัวที่อ้างถึงในคำอธิบายและในตารางสรุปอ่านมาจากภาพโดยตรง ภาพทั้งหมดเก็บไว้ที่ PS07/result/screenshots/ และลำดับคำสั่งที่ใช้ถ่ายภาพอยู่ใน PS07/docs/COMMANDS.md

1 ภาพรวมของโปรแกรม

1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน

ตัวอย่างในคาบเรียนใช้ thread หลายตัวบวกค่าตัวนับ (counter) ตัวเดียวกันไปในทิศทางเดียว รายงานนี้เลือกสถานการณ์ที่ต่างออกไปคือ **บัญชีเงินฝากร่วมหนึ่งบัญชี** ที่มีตู้ ATM หลายตู้ทำรายการพร้อมกัน โดยแต่ละตู้คือ thread หนึ่งตัว และยอดเงินในบัญชีคือตัวแปรที่ทุก thread ใช้ร่วมกัน

- บัญชีเปิดด้วยยอดเงิน 1,000 บาท (INITIAL_BALANCE)
- thread ครั้งแรกเป็น **ตู้ฝากเงิน** ฝากครั้งละ 1 บาท ส่วน thread ครั้งหลังเป็น **ตู้ถอนเงิน** ถอนครั้งละ 1 บาท ทุก thread ทำรายการเท่ากันคือ tx ครั้ง
- เงินที่ไหลเข้าจึงเท่ากับเงินที่ไหลออกเสมอ **ยอดเงินสุดท้ายที่ถูกต้องต้องเท่ากับ 1,000 บาท** ไม่ว่าจะใช้กี่ thread และไม่ว่าจะทำรายการกี่ครั้ง

สถานการณ์นี้ต่างจากตัวอย่างในคาบเรียนสามข้อ ข้อแรกคือมีการเปลี่ยนค่า**สองทิศทาง** (ทั้งเพิ่มและลด) ตามที่ใบงานกำหนดให้มีทั้ง increase และ decrease ผลของ race condition จึงออกมาได้ทั้งเป็นบวกและเป็นลบ ข้อที่สองคือความผิดพลาดมีความหมายชัดเจน ถ้ายอดสุดท้ายสูงกว่า 1,000 บาทแปลว่ารายการถอนบางรายการหายไป (ธนาคารเสียเงิน) ถ้าต่ำกว่าแปลว่ารายการฝากบางรายการหายไป (ลูกค้าเสียเงิน) และข้อที่สามคือโปรแกรม**ตรวจสอบตัวเองได้** เพราะแต่ละ thread นับจำนวนรายการที่ตัวเองทำเสร็จไว้ในตัวแปร local ที่ไม่มีใครใช้ร่วม จึงพิสูจน์ได้ว่ารายการทุกรายการถูกทำจริงครบถ้วน สิ่งที่มีแต่ยอดเงินในตัวแปรที่ใช้ร่วมกันเท่านั้น

1.2 โครงสร้างของโปรแกรม

ตัวแปรที่ทุก thread ใช้ร่วมกันมีเพียงตัวเดียวคือ balance ซึ่งเป็นตัวแปร global จึงอยู่ในส่วน data ของ process และทุก thread ใน process เดียวกันมองเห็นตำแหน่งหน่วยความจำเดียวกัน ส่วนข้อมูลของแต่ละตู้เก็บใน struct atm หนึ่งชุดต่อหนึ่ง thread

```

1 #define INITIAL_BALANCE 1000L
2 #define AMOUNT          1L
3 #define MAX_THREADS     64
4
5 long balance = INITIAL_BALANCE;
6
7 enum role { DEPOSITOR, WITHDRAWER, BOTH };
8
9 struct atm {
10     pthread_t tid;
11     enum role role;
12     long tx;           /* transactions this thread must make */
13     long deposits;     /* written once by the thread when done: */
14     long withdrawals; /* how many transactions it really made */
15 };

```

Code 1: ตัวแปรที่ใช้ร่วมกันและข้อมูลประจำตัวของแต่ละ thread

การฝากและถอนเขียนเป็นฟังก์ชันสั้น ๆ ที่แก้ค่า balance ตรง ๆ โดยไม่มี lock หรือกลไก synchronization ใด ๆ ป้องกันไว้ ซึ่งเป็นจุดที่โง่งนกำหนดให้มี (*without proper synchronization*)

```

1 void deposit_plain(void)
2 {
3     balance = balance + AMOUNT;
4 }
5
6 void withdraw_plain(void)
7 {
8     balance = balance - AMOUNT;
9 }

```

Code 2: การฝากและถอนเงินที่ไม่มีการป้องกัน

ฟังก์ชันที่ทุก thread รันคือ atm_thread() ซึ่งวนทำรายการตามบทบาทของตัวเอง ตัวนับ done_dep และ done_wdr เป็นตัวแปร local ที่อยู่บน stack ของแต่ละ thread แยกกัน จึงไม่มีทางเกิด race condition และค่าที่นับได้ถูกต้องเสมอ thread เขียนค่าเหล่านี้ก็กลับเข้า struct atm ของตัวเองเพียงครั้งเดียวตอนทำงานเสร็จ

```

1 void *atm_thread(void *arg)
2 {
3     struct atm *a = arg;
4     long done_dep = 0, done_wdr = 0;
5     long i;
6
7     if (a->role == DEPOSITOR || a->role == BOTH)
8         for (i = 0; i < a->tx; i++) {
9             deposit_plain();
10            done_dep++;
11        }
12     if (a->role == WITHDRAWER || a->role == BOTH)
13         for (i = 0; i < a->tx; i++) {
14             withdraw_plain();
15            done_wdr++;
16        }
17
18     a->deposits = done_dep;
19     a->withdrawals = done_wdr;
20     return NULL;
21 }

```

Code 3: ฟังก์ชันที่แต่ละ thread รัน

ฟังก์ชัน main() รับค่าจาก command line สองตัวคือจำนวน thread และจำนวนรายการต่อ thread (ค่าเริ่มต้นคือ 1 thread และ 1,000,000 รายการ คำสั่ง ./ps7 เหยย ๆ ตามใบงานจึงใช้ได้) จากนั้นกำหนดบทบาทของแต่ละ thread ตามตารางด้านล่าง สร้าง thread ทั้งหมดด้วย pthread_create() ก่อนแล้วจึงรอทุกตัวด้วย pthread_join() เพื่อให้ทุก thread ทำงานซ้อนกันได้ สุดท้ายรวมจำนวนรายการจากทุก thread มาคำนวณยอดที่ควรจะเป็น (expected) แล้วเทียบกับยอดจริง (actual)

Table 2: บทบาทของ thread ตามจำนวน thread ที่ระบุ

จำนวน thread	บทบาท
1	thread เดียวฝากเงิน tx ครั้งแล้วจึงถอน tx ครั้ง ใช้เป็นผลอ้างอิงแบบทำที่ละรายการตามลำดับ (Task 1)
เลขคู่ตั้งแต่ 2 ถึง 64	thread ครึ่งแรกฝากเงินอย่างเดียว ครึ่งหลังถอนเงินอย่างเดียว แต่ละตัวทำ tx รายการ (Task 2 และ Task 3)
เลขคี่ที่มากกว่า 1	ไม่รับ เพราะแบ่งฝากกับถอนให้เท่ากันไม่ได้ โปรแกรมพิมพ์วิธีใช้แล้วจบการทำงาน

แต่ละรอบการรันพิมพ์ผลเพียงบรรทัดเดียว เพื่อให้ภาพหน้าจอหนึ่งภาพแสดงผลของการรันซ้ำได้หลายรอบ ความหมายของแต่ละช่องเป็นดังนี้

```
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=... diff=... RACE
```

Code 4: รูปแบบผลลัพธ์หนึ่งบรรทัดต่อการรันหนึ่งรอบ

- threads และ tx คือจำนวน thread และจำนวนรายการต่อ thread ที่ระบุตอนรัน
- in และ out คือจำนวนรายการฝากและถอนทั้งหมดที่ thread นับได้จริง (จากตัวนับ local)
- expected คือยอดที่ควรเป็น คำนวณจาก $1000 + in - out$ ซึ่งเท่ากับ 1,000 เสมอ
- actual คือค่าใน balance หลังทุก thread ทำงานเสร็จ และ diff คือ $actual - expected$
- คำสุดท้ายเป็น ok เมื่อ diff เท่ากับศูนย์ และเป็น RACE เมื่อยอดเงินผิด

1.3 การคอมไพล์

คอมไพล์ด้วยคำสั่งตามใบงานทุกตัวอักษร คือ `gcc -o ps7 ps7.c -lpthread` ตัวเลือก `-lpthread` บอก linker ให้ลิงก์กับ POSIX thread library ส่วนการที่**ไม่ระบุระดับ optimization** (จึงเป็น `-O0`) เป็นความตั้งใจ เพราะถ้าใช้ `-O2` คอมไพเลอร์มีสิทธิ์มองว่าลูปที่บวก balance ทีละ 1 เป็นล่านครั้งมีผลเท่ากับการบวกครั้งเดียว และรวบทั้งลูปเหลือคำสั่งเดียว ช่วงเวลาที่ thread สองตัวจะชนกันจึงแทบหายไป ทั้งที่ได้ยังผิดอยู่เหมือนเดิม

2 Task 1: รันด้วย thread เดียว

ใบงานข้อ 2 ให้รันโปรแกรมด้วย thread เดียวก่อน เพื่อยืนยันว่าตรรกะของโปรแกรมถูกต้อง ถ้าผลที่ได้ในขั้นนี้ผิด ความผิดพลาดในขั้นต่อไปก็จะแยกไม่ออกว่าเกิดจาก race condition หรือเกิดจาก bug ธรรมดาในโปรแกรม

2.1 การคอมไพล์และสภาพแวดล้อม

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ nproc; gcc --version | head -1; gcc -o ps7 ps7.c -lpthread; ls -l ps7
22
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
-rwxrwxrwx 1 poomipat poomipat 16568 Sep 27 2026 ps7
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 1: จำนวน CPU รุ่นของ gcc และการคอมไพล์ด้วยคำสั่งตามใบงาน

จากภาพ คำสั่ง `nproc` ตอบ 22 คือ Linux มองเห็น CPU 22 ตัว ซึ่งหมายความว่า thread หลายตัวของโปรแกรมนี้อาศัย CPU คนละตัวและทำงานพร้อมกันได้จริง ข้อนี้จะมีผลมากใน Task 2 คอมไพเลอร์คือ gcc 13.3.0 ของ Ubuntu คำสั่ง `gcc -o ps7 ps7.c -lpthread` ไม่พิมพ์ข้อความใดออกมาเลย แปลว่าคอมไพล์ผ่านโดยไม่มีทั้ง error และ warning และ `ls -l` แสดงไฟล์ ps7 ขนาด 16,568 byte ที่มีสิทธิ์ execute (สิทธิ์ขึ้นเป็น `rw-rw-rw-rw` ทุกช่อง เพราะไฟล์อยู่บน drive C ของ Windows ที่ WSL2 mount ไว้ที่ `/mnt/c`)

2.2 ผลการรันด้วย thread เดียว

```
poomipat@DESKTOP-39EEJRM: ~$ for i in $(seq 5); do ./ps7 1 1000000; done
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 2: Task 1: รันด้วย thread เดียว 5 รอบ ได้ยอดเงินถูกต้องทุกรอบ

คำสั่ง `./ps7 1 1000000` สร้าง thread ทำงานเพียงตัวเดียว (thread main มีหน้าที่แค่สร้างแล้วรอด้วย `pthread_join()` และไม่แตะ `balance` จนกว่า thread ทำงานจะจบ) thread ตัวนี้ฝากเงิน 1,000,000 ครั้งจนยอดขึ้นเป็น 1,001,000 บาท แล้วจึงถอนอีก 1,000,000 ครั้งจนยอดกลับลงมา

ผลทั้ง 5 รอบในภาพเหมือนกันทุกตัวอักษร คือ `in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK` แสดงว่า

- ตรรกะของโปรแกรมถูกต้อง ทั้งการกำหนดขอบเขต การวนทำรายการ ตัวนับ local และการคำนวณยอดที่ควรเป็น
- ผลลัพธ์ **deterministic** คือรันกี่ครั้งก็ได้ค่าเดิม เพราะมี thread เพียงตัวเดียวที่อ่านและเขียน `balance` การอ่าน-บวก-เขียนแต่ละครั้งจึงจบครบก่อนเริ่มครั้งถัดไปเสมอ
- แม้ระบบปฏิบัติการจะขัดจังหวะ thread ตัวนี้กลางคันเพื่อไปรันงานอื่นได้ตลอดเวลา แต่ค่าที่ค้างอยู่ใน register ถูกเก็บไว้กับ context ของ thread และถูกคืนกลับมาครบเมื่อได้ทำงานต่อ และระหว่างนั้นไม่มี thread อื่นแก้ `balance` ค่าที่เขียนกลับจึงยังถูกต้อง

ผลนี้เป็นเส้นฐานสำหรับเทียบกับขั้นต่อไป ถ้าใช้หลาย thread แล้วยอดเงินผิดไปจาก 1,000 บาท สาเหตุจึงไม่ใช่ bug ในตรรกะของโปรแกรม แต่มาจากการที่หลาย thread แก้วตัวแปรเดียวกันพร้อมกันเท่านั้น

3 Task 2: หลาย thread โดยไม่บังคับ context switch

ใบงานข้อ 3 ให้แก้โปรแกรมให้สร้าง thread มากกว่าหนึ่งตัว และข้อ 4 ให้รันโดยไม่บังคับ context switch หลาย ๆ ครั้งเพื่อดูว่าผลไม่แน่นอน โปรแกรมนี้รับจำนวน thread จาก command line อยู่แล้ว การแก้ไขครั้งนี้จึงเป็นการสั่ง `./ps7 4 1000000` แทน `./ps7 1 1000000` ซึ่งทำให้ `main()` สร้าง thread 4 ตัว เป็นตู้ฝากเงิน 2 ตัวและตู้ถอนเงิน 2 ตัว แต่ละตัวทำ 1,000,000 รายการ รวมเป็นฝาก 2,000,000 ครั้งและถอน 2,000,000 ครั้ง ยอดที่ถูกต้องจึงยังเป็น 1,000 บาทเหมือนเดิม ตัวฟังก์ชัน `deposit_plain()` และ `withdraw_plain()` ไม่ได้แก้ไขแม้แต่บรรทัดเดียว

การทดลองนี้รันสองแบบ แบบแรกปล่อยให้ระบบปฏิบัติการจัดวาง thread บน CPU ทั้ง 22 ตัวตามปกติ แบบที่สองใช้ `taskset -c 0` บังคับให้ทุก thread อยู่บน CPU หมายเลข 0 ตัวเดียว เพื่อแยกให้เห็นว่าผลที่ผิดเกิดจากกลไกใด

3.1 รันบน CPU 22 ตัว

```
poomipat@DESKTOP-39EEJRM: ~$ for i in $(seq 10); do ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-32229 diff=-33229 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-130619 diff=-131619 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=152202 diff=+151202 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=121915 diff=+120915 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=140707 diff=+139707 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-6078 diff=-7078 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-113785 diff=-114785 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-725602 diff=-726602 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=380062 diff=+379062 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-186190 diff=-187190 RACE
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 3: Task 2: 4 thread บน CPU 22 ตัว รัน 10 รอบ ผิดทุกรอบและไม่ซ้ำกันเลย

ทั้ง 10 รอบในภาพลงท้ายด้วย RACE ค่า diff ของแต่ละรอบเรียงตามลำดับคือ -33,229, -131,619, +151,202, +120,915, +139,707, -7,078, -114,785, -726,602, +379,062 และ -187,190 ข้อสังเกตจากภาพมีดังนี้

- ผลไม่ซ้ำกันเลยสักรอบ ทั้งที่เป็นโปรแกรมเดียวกัน รับ argument เดียวกัน และรันบนเครื่องเดียวกันติดต่อกัน นี่คือพฤติกรรมแบบ non-deterministic ที่โปรแกรมเมอร์ต้องระวัง
- ผิดได้ทั้งสองทิศทาง มีรอบที่ยอดเกิน 4 รอบ (รายการถอนหายไปมากกว่ารายการฝาก ธนาคารเสียเงิน) และรอบที่ยอดขาด 6 รอบ (รายการฝากหายไปมากกว่า ลูกค้าเสียเงิน) รอบที่ผิดมากที่สุดยอดเงินขาดไป 726,602 บาท และรอบที่ใกล้เคียงที่สุดก็ยิ่งผิด 7,078 บาท
- รายการครบทุกรายการ ช่อง in=2000000 out=2000000 เท่ากันทุกบรรทัด แปลว่าทุก thread ทำรายการครบตามจำนวนจริง สิ่งที่ยกไปคือผลของการเขียนลงตัวแปร balance ไม่ใช่ตัวรายการ
- ค่า diff เป็นเพียงผลสุทธิระหว่างรายการฝากที่หายกับการถอนที่หาย ซึ่งหักล้างกันได้ จำนวนรายการที่หายไปจริงจึงมากกว่าหรือเท่ากับค่าสัมบูรณ์ของ diff เสมอ

สาเหตุที่ผิดทุกรอบในกรณีนี้คือเครื่องมี CPU มากพอให้ thread ทั้ง 4 ตัวได้ CPU คนละตัว thread จึงทำงานพร้อมกันจริง (parallelism) ไม่ต้องรอให้ระบบปฏิบัติการสลับให้ ในทุก ๆ รอบของ loop CPU สองตัวมีโอกาสอ่านค่า balance ค่าเดียวกันไปพร้อมกัน แล้วต่างคนต่างเขียนผลของตัวเองกลับ ผลของตัวที่เขียนก่อนจึงถูกเขียนทับหายไป เมื่อลูวนเป็นล้านรอบ การชนแบบนี้จึงเกิดขึ้นเป็นจำนวนมหาศาล และจำนวนที่ชนในแต่ละรอบการรันขึ้นอยู่กับจังหวะเวลาที่ควบคุมไม่ได้ เช่น thread ไหนเริ่มก่อน ได้ CPU ตัวใด และ CPU ตัวอื่นกำลังทำอะไรอยู่

3.2 รันบน CPU ตัวเดียวด้วย taskset

```
poomipat@DESKTOP-39EEJRM: ~$ for i in $(seq 20); do taskset -c 0 ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=2001000 diff=+2000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-628150 diff=-629150 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 4: Task 2: 4 thread บน CPU ตัวเดียว รัน 20 รอบ ถูก 13 รอบ ผิด 7 รอบ

เมื่อบังคับให้ทุก thread อยู่บน CPU ตัวเดียว ในแต่ละขณะจะมี thread ทำงานได้เพียงตัวเดียว thread จะสลับกันก็ต่อเมื่อ kernel ขัดจังหวะ thread ที่กำลังทำงาน (timer interrupt เมื่อหมด time slice) แล้วเลือก thread อื่นขึ้นมาทำงานแทน ผลในภาพต่างจากกรณี CPU 22 ตัวอย่างชัดเจน

- ถูก 13 รอบ ผิด 7 รอบ รอบที่ผิดคือรอบที่ 3, 6, 8, 12, 17, 19 และ 20 ส่วนที่เหลือได้ actual=1000 diff=+0 OK
- เมื่อผิดจะผิดเป็นก้อนใหญ่ที่ลงตัว รอบที่ 6, 8, 17, 19 และ 20 ได้ diff=+1000000 พอดี ซึ่งเท่ากับงานทั้งหมดของ thread ถอนเงินหนึ่งตัว รอบที่ 3 ได้ diff=+2000000 เท่ากับงานของ thread ถอนเงินทั้งสองตัว และรอบที่ 12 ได้ diff=-629150 คือรายการฝาก 629,150 รายการหายไป

รูปแบบนี้อธิบายได้ด้วยจังหวะของ context switch ลูปของ thread หนึ่งตัว (1,000,000 รอบ) ใช้เวลาเพียงไม่กี่มิลลิวินาที thread ส่วนใหญ่จึงทำงานจบได้โดยไม่ต้อง time slice และถึงจะโดนขัดจังหวะ จุดที่โดนขัดก็มักไม่ตกอยู่ระหว่างการอ่านกับการเขียน balance รอบส่วนใหญ่จึงถูกต้อง แต่ถ้าบังเอิญ thread ฝากเงินตัวหนึ่งโดนขัดจังหวะหลังจาก

อ่านค่า `balance` เข้า `register` แล้วแต่ยังไม่ได้เขียนกลับ ค่าใน `register` นั้นจะถูกเก็บไว้กับ `context` ของมัน ระหว่างที่รอ `thread` ถอนเงินได้ขึ้นมาทำงานและถอนครบทั้ง 1,000,000 ครั้ง เมื่อ `thread` ฝากเงินได้กลับมาทำงานต่อ มันเขียนค่าเก่าที่ค้างอยู่ทับลงไป รายการถอนทั้งล้านรายการจึงหายไปในการเขียนครั้งเดียว ได้ `diff=+1000000` พอดี กรณี `diff=-629150` ก็เป็นกลไกเดียวกันในทิศตรงข้าม คือ `thread` ถอนเงินถือค่าเก่าค้างไว้ขณะที่ `thread` ฝากเงินทำไปได้ 629,150 รายการ

นี่คือ **untimely context switch** ตามที่ใบงานอธิบาย ซึ่งเกิดขึ้นได้เองตามธรรมชาติแต่ไม่บ่อย จึงต้องรันหลายครั้งถึงจะเห็น และเป็นเหตุผลที่ Task 3 ต้องบังคับให้เกิด

3.3 ทำไม `balance = balance + AMOUNT` จึงไม่ปลอดภัย

คำสั่ง C หนึ่งบรรทัดดูเหมือนทำงานเสร็จในขั้นเดียว แต่ CPU ไม่ได้บวกค่าในหน่วยความจำโดยตรง ภาพด้านล่างใช้ `objdump` แปลงไฟล์ `ps7` ตัวเดียวกับที่ใช้รันใน `t03` และ `t04` กลับเป็นภาษา `assembly` (รูปแบบ Intel ที่เขียนปลายทางไว้ก่อนต้นทาง)

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ objdump -d --no-show-raw-insn -M intel ps7 | awk '/<deposit|withdraw> plain>:/,/ret/'
000000000001269 <deposit_plain>:
1269:    endbr64
126d:    push    rbp
126e:    mov     rbp, rsp
1271:    mov     rax, QWORD PTR [rip+0x2d98]    # 4010 <balance>
1278:    add     rax, 0x1
127c:    mov     QWORD PTR [rip+0x2d8d], rax    # 4010 <balance>
1283:    nop
1284:    pop     rbp
1285:    ret
000000000001286 <withdraw_plain>:
1286:    endbr64
128a:    push    rbp
128b:    mov     rbp, rsp
128e:    mov     rax, QWORD PTR [rip+0x2d7b]    # 4010 <balance>
1295:    sub     rax, 0x1
1299:    mov     QWORD PTR [rip+0x2d70], rax    # 4010 <balance>
12a0:    nop
12a1:    pop     rbp
12a2:    ret
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 5: คำสั่งเครื่องของ `deposit_plain()` และ `withdraw_plain()`

ทั้งสองฟังก์ชันมีโครงสร้างเหมือนกัน คำสั่งที่เหลือเป็นเพียงการเข้าและออกจากฟังก์ชัน ส่วนที่เป็นหัวใจของ `race condition` คือสามคำสั่งตรงกลาง ซึ่ง `objdump` ระบุไว้ท้ายบรรทัดว่าอ้างอิงตัวแปร `<balance>` ที่ตำแหน่ง 4010 เดียวกัน

Table 3: สามขั้นตอนของการแก้ค่า `balance` ที่อ่านได้จากภาพ

ขั้น	ที่อยู่	คำสั่ง	ความหมาย
Load	1271 / 128e	<code>mov rax, QWORD PTR [...]</code>	อ่านค่า <code>balance</code> จากหน่วยความจำมาไว้ใน <code>register rax</code>
Modify	1278 / 1295	<code>add rax, 0x1</code> / <code>sub rax, 0x1</code>	บวกหรือลบ 1 เฉพาะใน <code>register</code> หน่วยความจำยังเป็นค่าเดิม
Store	127c / 1299	<code>mov QWORD PTR [...], rax</code>	เขียนค่าใน <code>rax</code> กลับลง <code>balance</code> ทับค่าที่อยู่ในหน่วยความจำขณะนั้น

ปัญหาอยู่ที่ช่วงเวลาระหว่าง Load กับ Store ถ้ามี `thread` อื่นแก้ `balance` ในช่วงนี้ ไม่ว่าจะเพราะทำงานพร้อมกันบน CPU อีกตัว (`t03`) หรือเพราะ `thread` นี้ถูกสลับออกไปกลางคัน (`t04`) ขั้น Store จะเขียนค่าที่คำนวณจากข้อมูลเก่าทับลงไป และการแก้ของ `thread` อื่นจะหายไปโดยไม่มีสัญญาณเตือนใด ๆ การอ่าน-แก้-เขียน (`read-modify-write`) สามขั้นนี้จึงไม่เป็น **atomic** และโค้ดส่วนนี้คือ `critical section` ที่ไม่มีการป้องกัน

3.4 เปรียบเทียบผลบน CPU 22 ตัวกับ CPU ตัวเดียว

Table 4: เปรียบเทียบผลของ Task 2 ทั้งสองแบบ (อ่านจากภาพ t03 และ t04)

ประเด็น	CPU 22 ตัว (t03)	CPU ตัวเดียว (t04)
จำนวนรอบที่ผิด	10 จาก 10 รอบ	7 จาก 20 รอบ
ลักษณะของค่าที่ผิด	ไม่ซ้ำกันเลย มีทั้งบวกและลบ ตั้งแต่ -726,602 ถึง +379,062	ลงตัวเป็นก้อนได้แก่ +1,000,000 จำนวน 5 รอบ, +2,000,000 และ -629,150 อย่างละ 1 รอบ
ลักษณะการทำงาน	พร้อมกันจริงบน CPU คนละตัว	ทีละตัว สลับกันเมื่อ kernel ชัดจังหวะ
การชนกันเมื่อใด	ได้ทุกรอบของกลุ่ม เมื่อ CPU สองตัวเข้าช่วง Load-Store พร้อมกัน	เฉพาะเมื่อ context switch ต่กระหว่าง Load กับ Store พอดี

ทั้งสองแบบใช้โปรแกรมเดียวกันและผิดด้วยสาเหตุเดียวกัน คือการอ่าน-แก้-เขียนตัวแปรที่ใช้ร่วมกันโดยไม่มีการป้องกัน ต่างกันเพียงว่าอะไรเป็นตัวแทรกเข้ามาในช่วง Load-Store บนเครื่องที่มี CPU หลายตัว การทำงานพร้อมกันจริงทำให้ race condition ปรากฏแทบทุกครั้ง ส่วนบน CPU ตัวเดียว race condition จะปรากฏเฉพาะเมื่อ context switch เกิดขึ้นผิดจังหวะ ซึ่งเป็นเหตุการณ์ที่สุ่มและไม่บ่อย โปรแกรมที่มี bug แบบนี้จึงอาจผ่านการทดสอบหลายครั้งแล้วค่อยพังในภายหลัง